

Natural Language Processing and Transformers

October 30, 2025

Columbia University

Tutorial 6. COMS 4996



The Big Picture

- **Natural language** (& speech) are central to human intelligence
- **Natural Language Processing** attempts to capture (some of) this intelligence algorithmically and is of huge practical importance
 - machine translation, chatbots, automatic fact checking, etc.
- NLP has seen several transformative shifts:
 - "statistical" NLP → word embeddings (~2013, e.g. **word2vec**)
 - word embeddings → pre-trained language models (~2017, e.g. **BERT**)
 - purely self-supervised → SSL + SFT + RLHF (~2022, e.g. **ChatGPT**)

The Big Picture

- Natural language (& speech) are central to human intelligence
- **N**atural **L**anguage **P**rocessing (**NLP**) attempts to capture some of this intelligence algorithmically and is of huge practical importance
 - machine translation, chatbots, automatic fact checking, etc.
- NLP has seen several transformative shifts:

Today... “statistical” NLP → word embeddings (~2013, e.g. **word2vec**)

- word embeddings → pre-trained language models (~2017, e.g. **BERT**)
- purely self-supervised → SSL + SFT + RLHF (~2022, e.g. **ChatGPT**)

The Big Picture

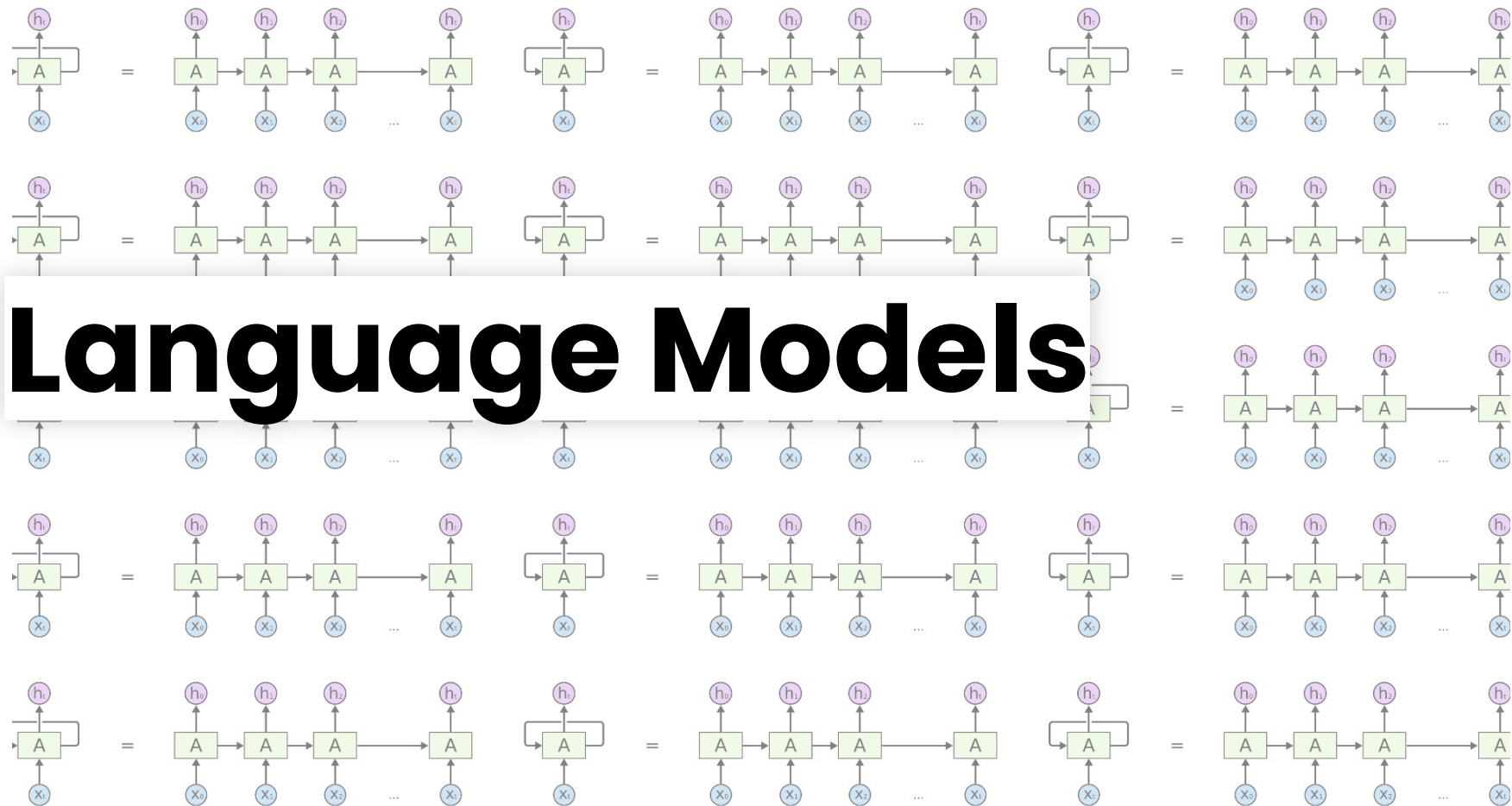
- Natural language (& speech) are central to human intelligence
- **N**atural **L**anguage **P**rocessing (**NLP**) attempts to capture some of this intelligence algorithmically and is of huge practical importance
 - machine translation, chatbots, automatic fact checking, etc.
- NLP has seen several transformative shifts:
 - “statistical” NLP → word embeddings (~2013, e.g. **word2vec**)

Next tutorial... embeddings → pre-trained language models (~2017, e.g. **BERT**)

- purely self-supervised → SSL + SFT + RLHF (~2022, e.g. **ChatGPT**)

Goals of this Tutorial

- Build basic NLP literacy by looking at **language models**
- Get up to date with recent developments in NLP
 - **BERT, GPT, Self-Supervised Learning (SSL)**
- Know where to look if you're starting an NLP project
- *Will focus more on building intuition than math*



Language Models

Language models (LMs) assign **probabilities** to sequences of words

- **Speech recognition:** $P(\text{I will be back soonish}) > P(\text{I will be bassoon dish})$
- **Spell checkers:** $P(\text{There are two midterms}) > P(\text{Their are two midterms})$
- **Machine translation:**

他 向 记者 介绍了 主要 内容

$P(\text{He to reporters introduced main content}) <$

$P(\text{He briefed reporters on the main contents of the statement})$

Anatomy of a Language Model

Is the product of the **conditional probability of each word and its history** (chain rule)

$$P(w_{1:n}) = \prod_{k=1}^n P(w_k | w_{<k})$$

The probability of a sequence of n words

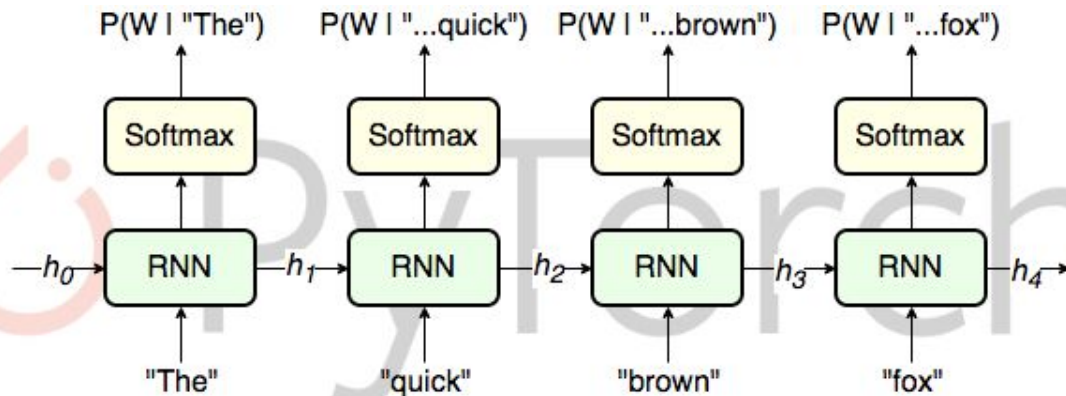
$$= \sum_{k=1}^n \log P(w_k | w_{<k})$$

How do we compute the **conditional probability**?

In practice, we take the log sum

RNNs as Language Models

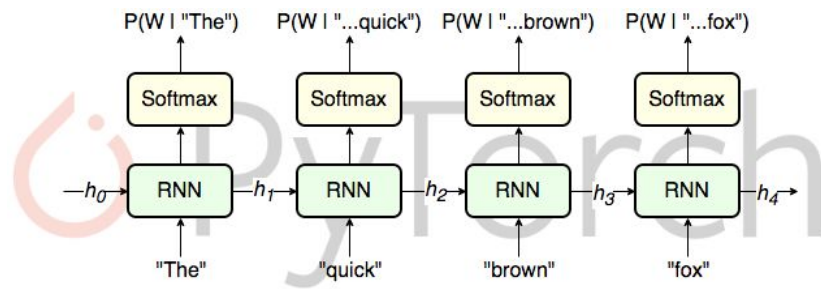
- We can use **recurrent neural networks** (RNNs) like LSTMs
- Work well for variable length inputs, like sentences



Shortcomings of RNNs

RNNs have some shortcomings:

- Difficulty modelling long range dependencies
 - Backpropagation on a 100 word sequence $\sim$ 100 layer network!
- Not parallelizable within training examples
 - Doesn't take full advantage of GPUs



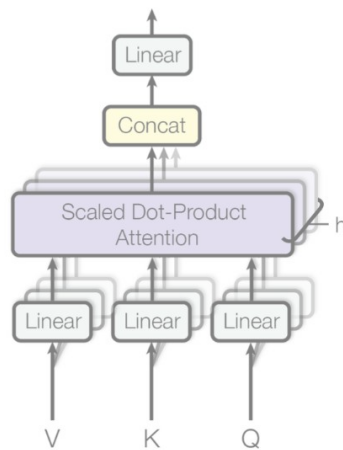
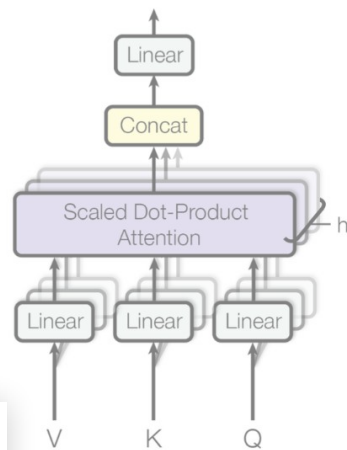
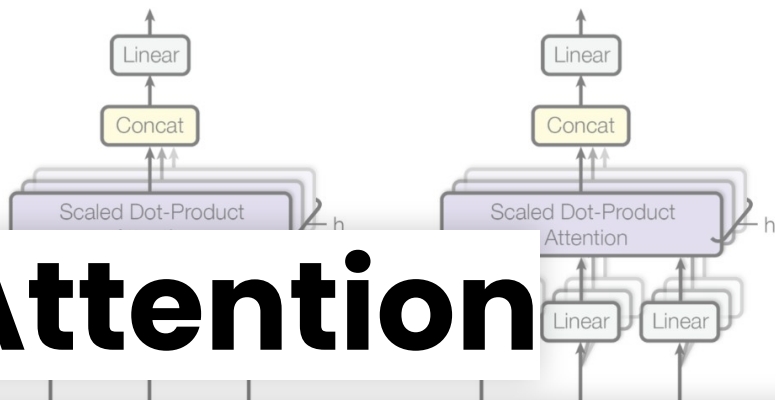
Shortcomings of RNNs

We'd like an "architectural primitive" that...

- Is parallelizable within training examples
 - Take advantage of accelerators like GPUs/TPUs
- Directly facilitates interactions between timesteps/tokens
 - To better model long range dependencies

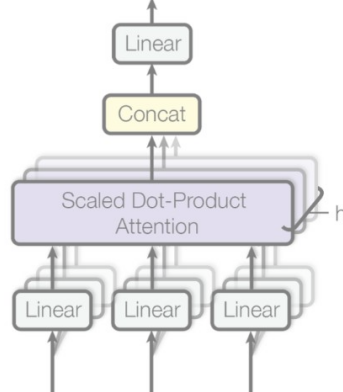
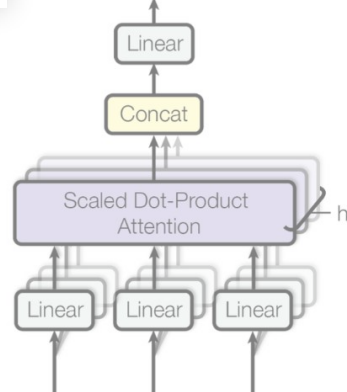
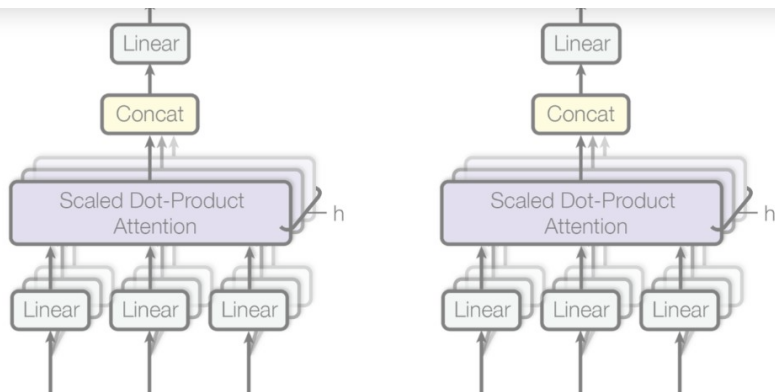
Replacing **recurrence** with **attention** will give us both of these!

Attention Is All You Need

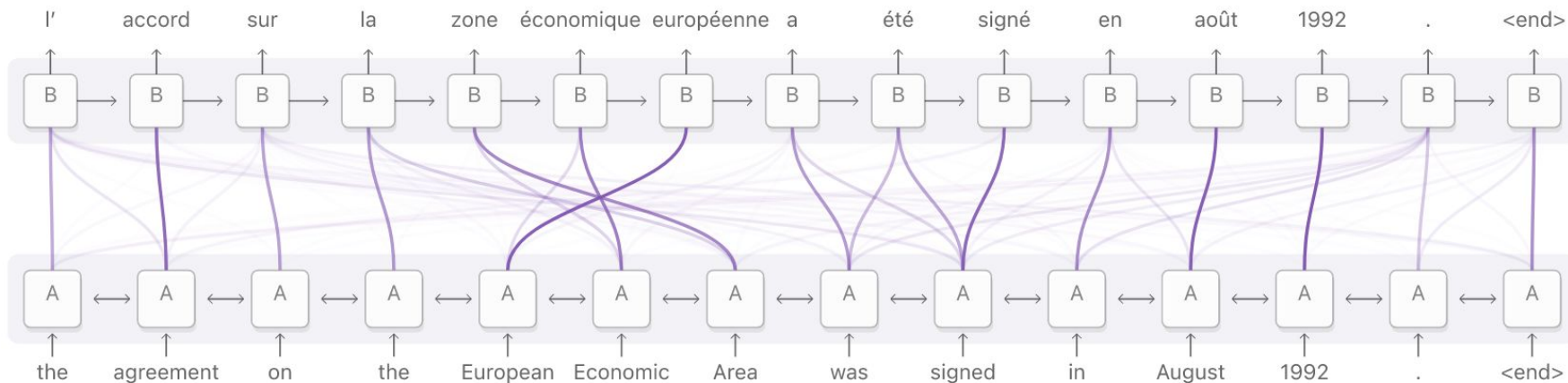


Multi-Head Attention

Multi-Head Attention



Attention Primer: Machine Translation

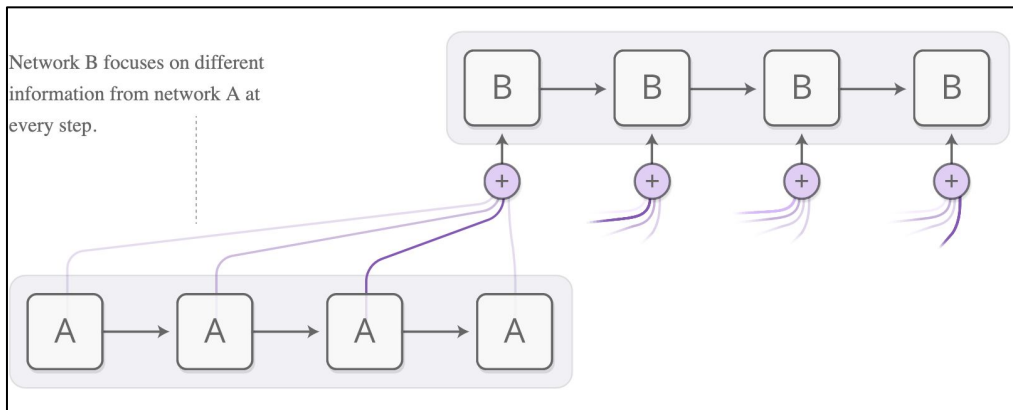


You can't cram the meaning of a whole %&!\$# sentence into a single \$&!# vector!*

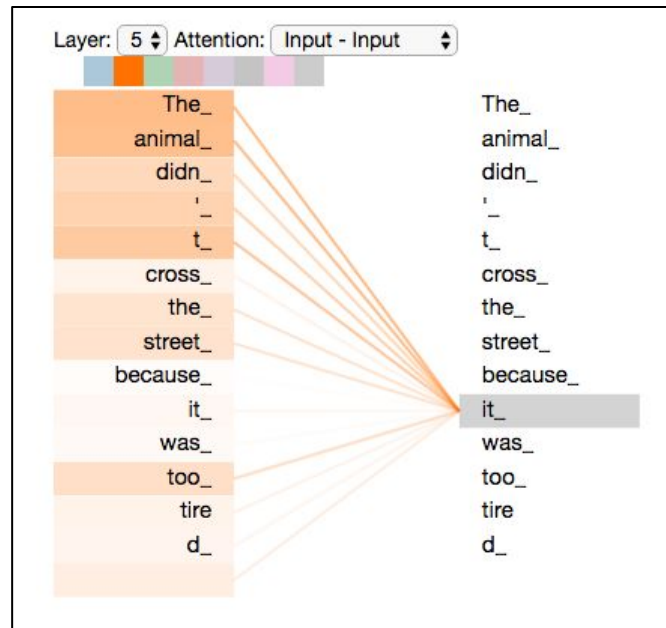
Replacing Recurrence with Attention

Why stop there? Why not *replace recurrence with attention!*

1 Cross-attention



2 Self-attention



Attention Is All You Need

- In the [AIAYN paper](#), recurrence was *completely replaced* with attention in the **transformer architecture**
- Introduced a new type of *self*-attention, (**scaled**) **dot-product** attention
- Like any attention mechanism, we need to make two decisions:
 - How to compute similarity/compatibility? → **dot-product**
 - How do we normalize the similarity score? → **softmax**

Anatomy of Scaled Dot-Product Attention

Scaled dot-product attention
takes three matrices as input

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

A softmax normalizes
similarities $\rightarrow [0, 1]$

Similarity is simply the dot
product between Q and K

The output is simply
a scaling of V

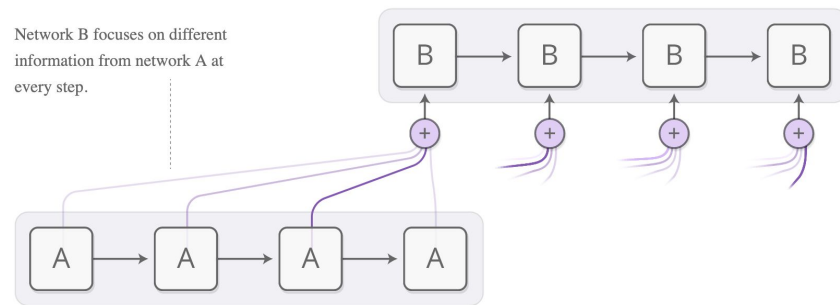
Attention maps a query, Q and a set of key-value (K, V) pairs to an output. The output is **computed** as a **weighted sum** of the **values**, where the weight assigned to each value is computed by a **compatibility function** of the **query** with the **corresponding key**.

Queries, Keys and What?

Depend on *how the attention mechanism is used*

- In **cross**-attention, $K == V$ and come from the *encoder*, Q comes from the *decoder*

The decoder "focuses" on certain tokens in the encoders output



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

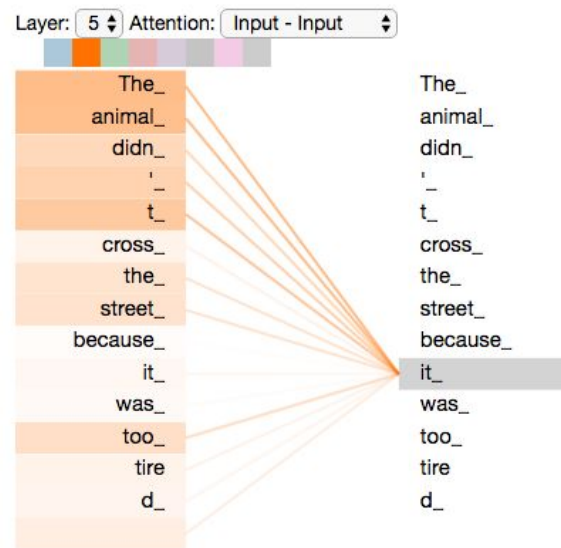
Queries, Keys and What?

Depend on *how the attention mechanism is used*

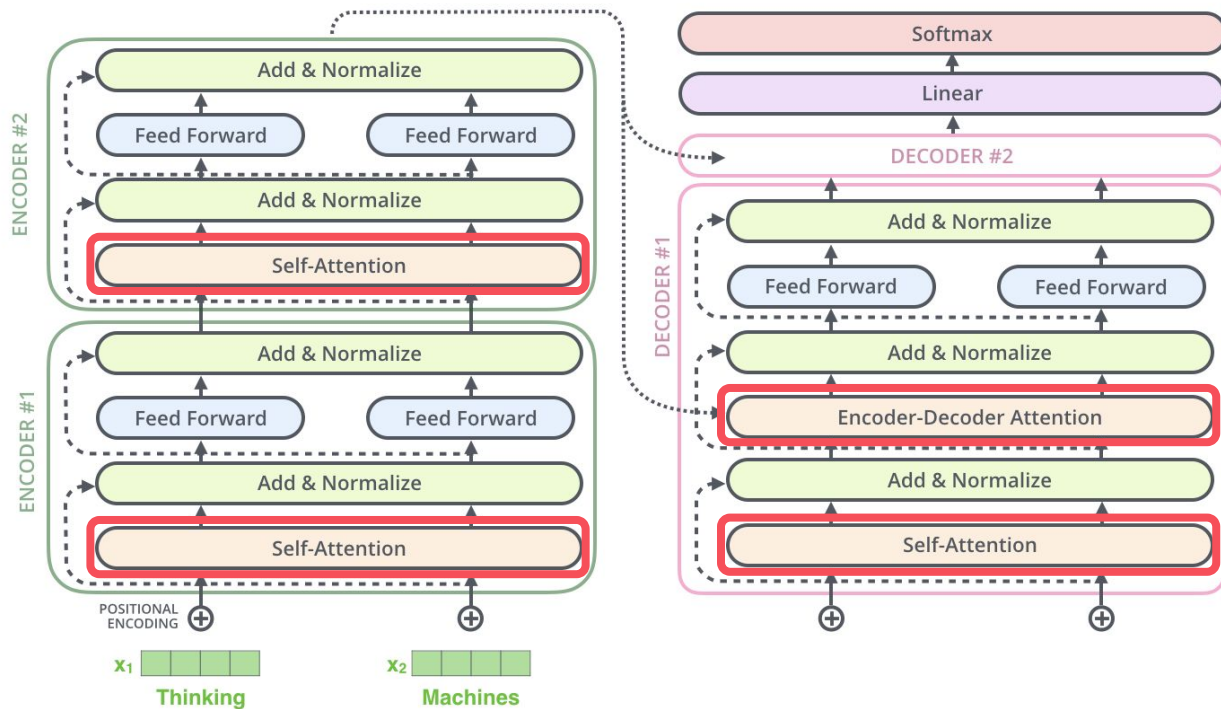
- In **self**-attention, $Q == K == V ==$ the token representations of the input sequence

Updating the representation of each token based on the other tokens in the sequence

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

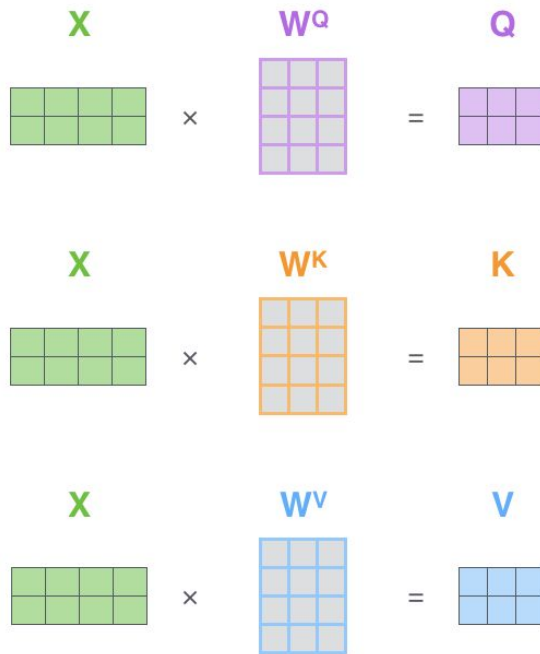


Transformers (covered in lecture)



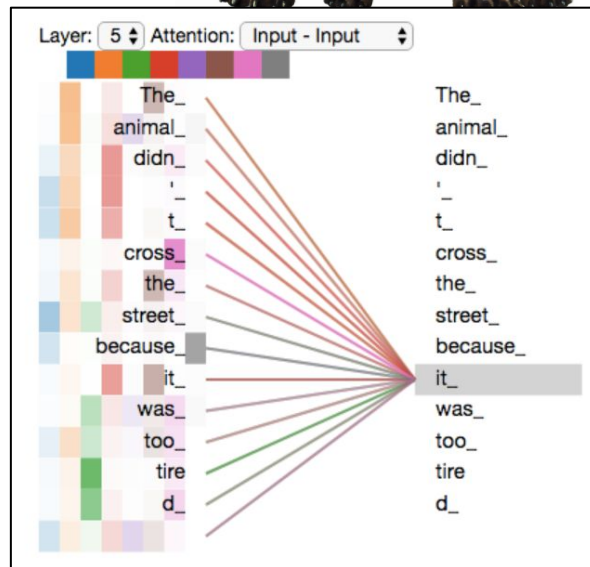
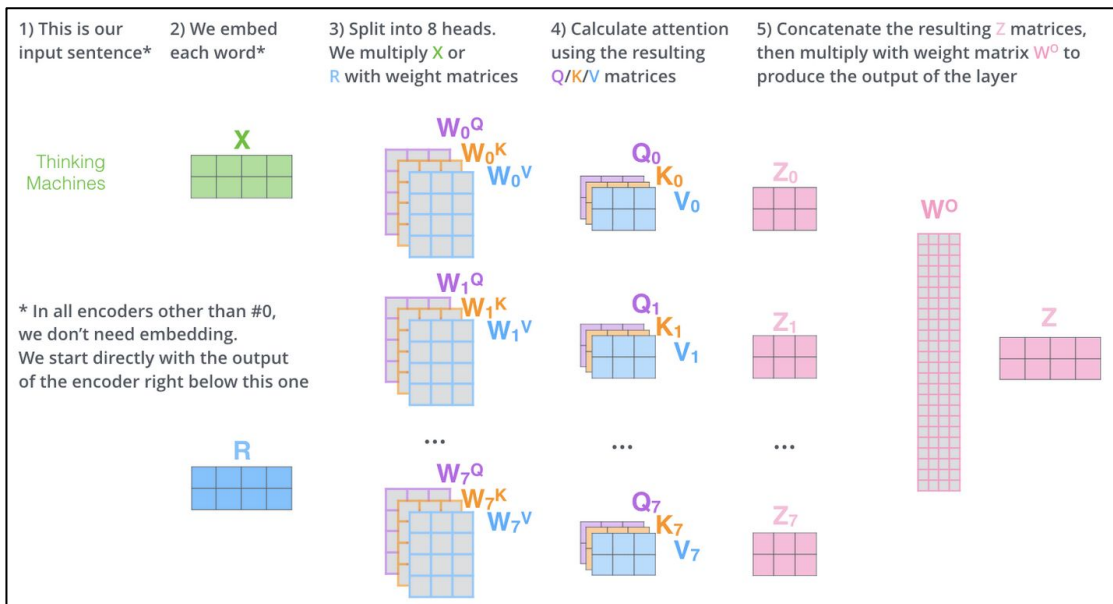
Projections

- Q, K & V are *projections of embedded tokens*
- Contain (most off) the **learnable** params of SDPA!
- In a multi-layered network (e.g. a transformer), they are outputs of the previous layer



The Beast with Many Heads

Usually, we use **multi-head** scaled dot-product attention



Different Heads Capture Different Patterns

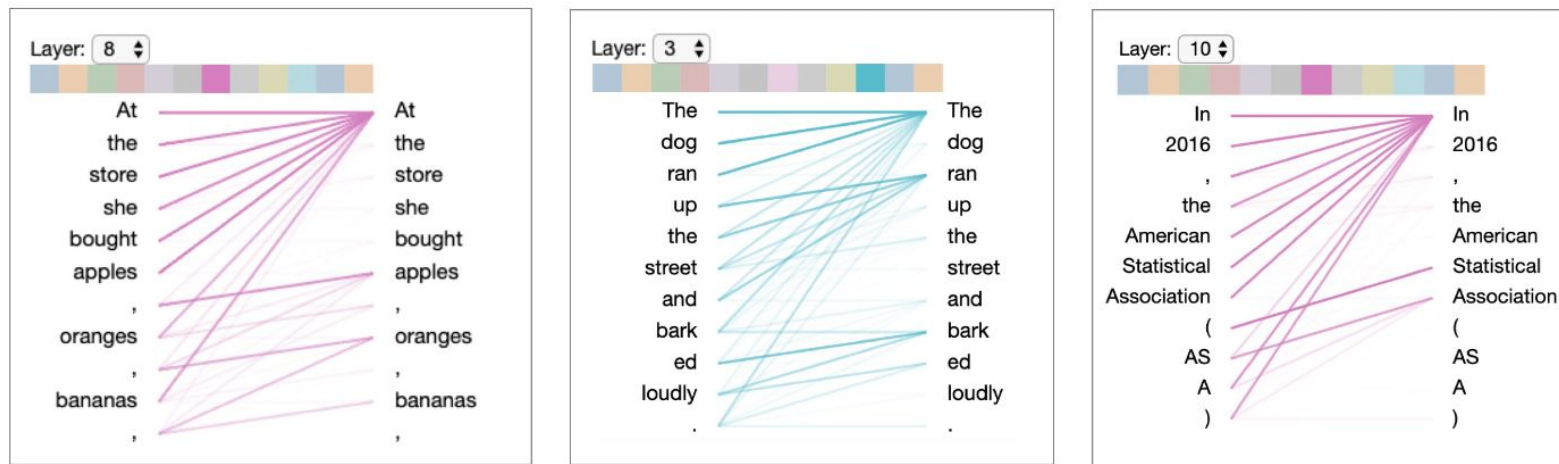


Figure 3: Examples of attention heads in GPT-2 that capture specific lexical patterns: list items (left); verbs (center); and acronyms (right). Similar patterns were observed in these attention heads for other inputs. Attention directed toward first token is likely null attention ([Vig and Belinkov, 2019](#)).

The Payoff

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

Before we move on...

- **LMs** assign *probabilities to sequences* and are the “workhorse” of NLP
- Typically implemented with RNNs; largely replaced by **Transformers**
- **Multi-head scaled dot-product** attention the backbone of Transformers
 - Easier to learn long range dependencies
 - Parallelize computation within training examples

How do we train Transformers as language models?



Pretrained

Language Models &

Self-Supervised Learning



Train Transformers as Language Models

- Learn general properties of language → *transfer to downstream tasks!*
- Take advantage of transformers *efficiency* → train on a **huge** amount of data!
- (At least) two paradigms have emerged:
 - **Generative Pretrained Transformer (GPT)** → Next-token prediction, decoder only transformer
 - **Bidirectional Encoder Representations from Transformers (BERT)** → Masked language modelling, encoder only transformer
- Both are trained with **Self-Supervised Learning (SSL)**

Self Supervised Learning

Three types of learning

- Supervised learning
- Reinforcement learning
- Unsupervised/self-supervised learning
 - When the label is the data itself
 - Possible to make use of large amounts of data with no additional labeling effort

Examples of Self Supervised Learning

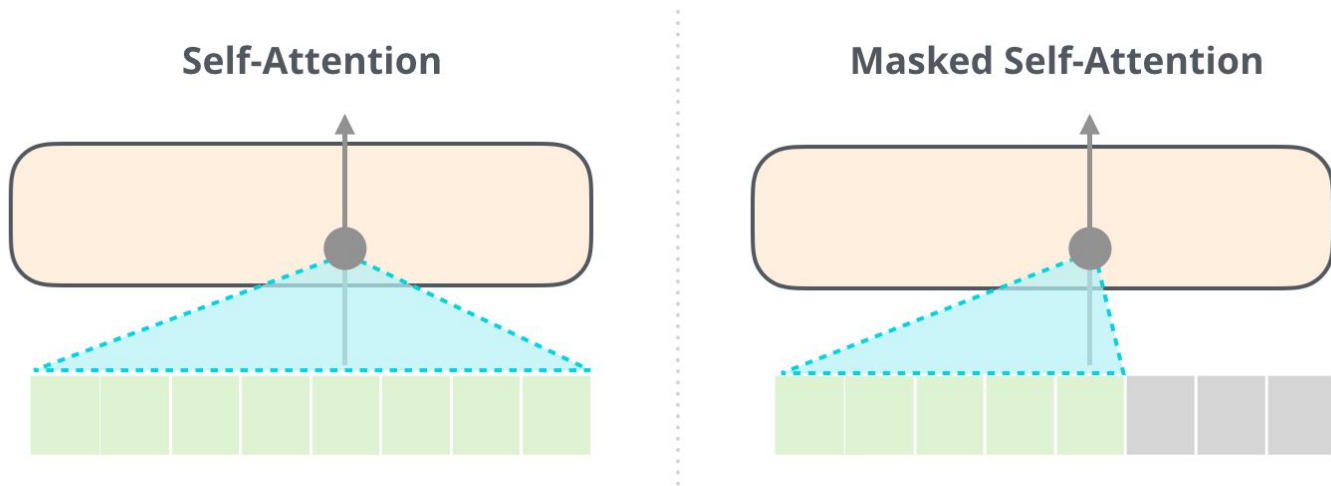
- Predict next frame in a video
- Image Completion
- Auto-encoding tasks
- Rotation prediction
- **Predicting the next word from the previous ones**

Generative Pretrained Transformer (GPT)

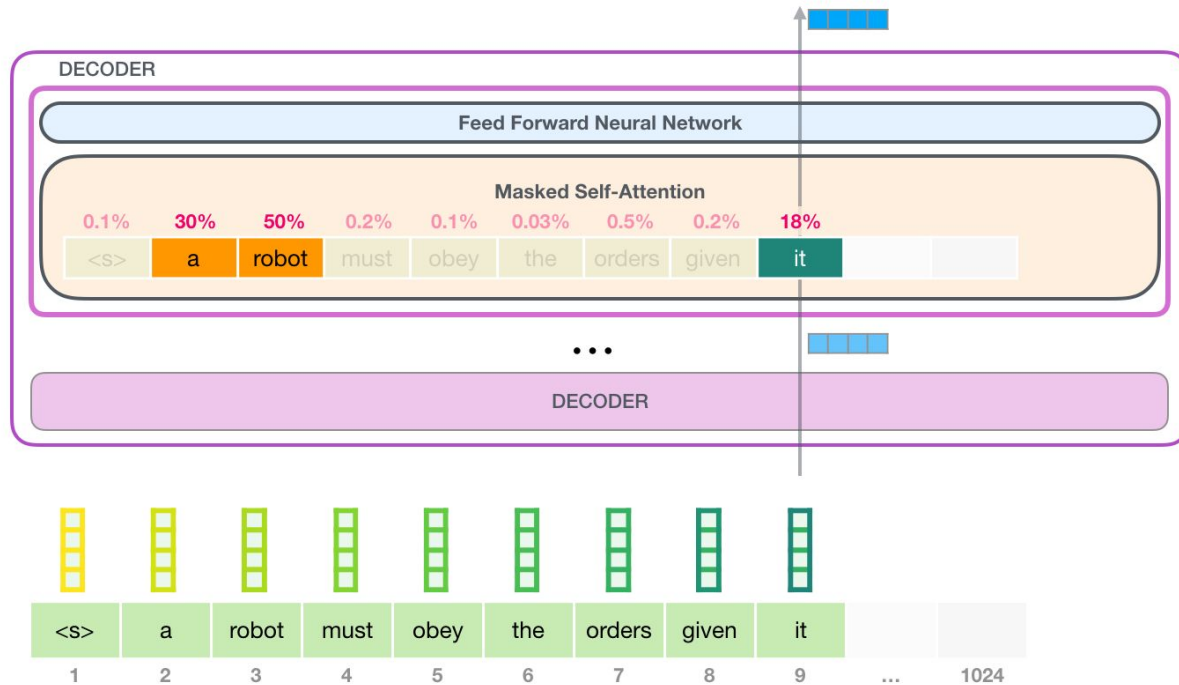
- **GPT** is a *decoder only* transformer pretrained on huge amounts of text
- GPT-3 is trained on 45TB of unlabelled text, ~410 billion tokens
- The (pre)training objective is simply to predict the next token
- For this, we will need to slightly tweak the self-attention...

Masked Self-Attention

Future timesteps are **masked** to prevent decoder from “peeking”



Next Token Prediction



GPT can be used for any “text in, text out” task



Mood to color

Transformation

Generation

Turn a text description into a color.

Prompt

The CSS code for a color like a blue sky at dusk:

background-color: #

Sample response

B2CED1



SQL request

Transformation

Generation

Translation

Create simple SQL queries.

Prompt

Create a SQL request to find all users who live in California and have over 1000 credits:

Sample response

```
SELECT * FROM users WHERE state='CA' AND credits > 1000;
```


GPT can be used for any “text in, text out” task



Grammar correction

Transformation

Generation

Corrects sentences into standard English.

Prompt

Correct this to standard English:

She no went to the market.

Sample response

She didn't go to the market.



English to other languages

Transformation

Generation

Translates English text into French, Spanish and Japanese.

Prompt

Translate this into 1. French, 2. Spanish and 3. Japanese:

What rooms do you have available?

1.

Sample response

Quels sont les chambres disponibles?

2. ¿Cuáles son las habitaciones disponibles?

3. 何室がありますか?

Bidirectional Encoder Representations from Transformers (BERT)

- GPT is a **unidirectional** LM, incorporating context from *previous* tokens
- This is likely sub-optimal for many token- or sentence-level tasks
- BERT is a **bidirectional** LM based on a transformer encoder
- BERT is pretrained with two self-supervised objectives:
 - **M**asked **L**anguage **M**odelling (**MLM**)
 - **N**ext **S**entence **P**rediction (**NSP**)

Bidirectional Encoder Representations from Transformers (BERT)

- GPT is a **unidirectional** LM, incorporating context from *previous* tokens
- This is likely sub-optimal for many token- or sentence-level tasks

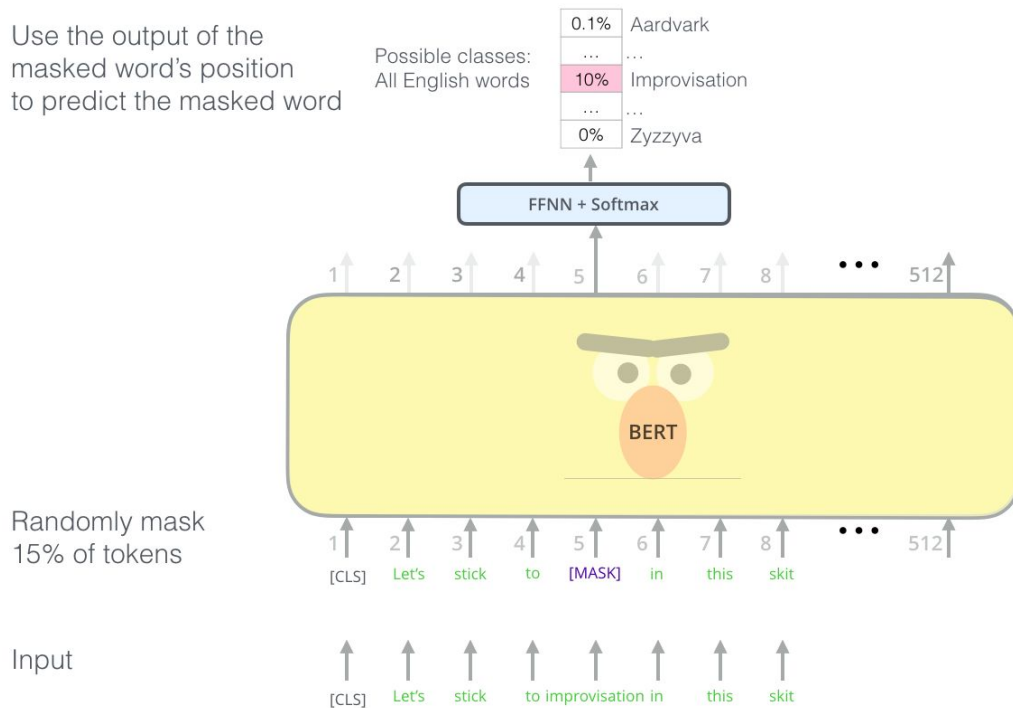
Turns out NSP isn't useful! We will focus on MLM mer encoder

- BERT is pretrained with two self-supervised objectives:
 - **M**asked **L**anguage **M**odelling (**MLM**)
 - ~~**N**ext **S**entence **P**rediction (**NSP**)~~



Masked Language Modelling (MLM)

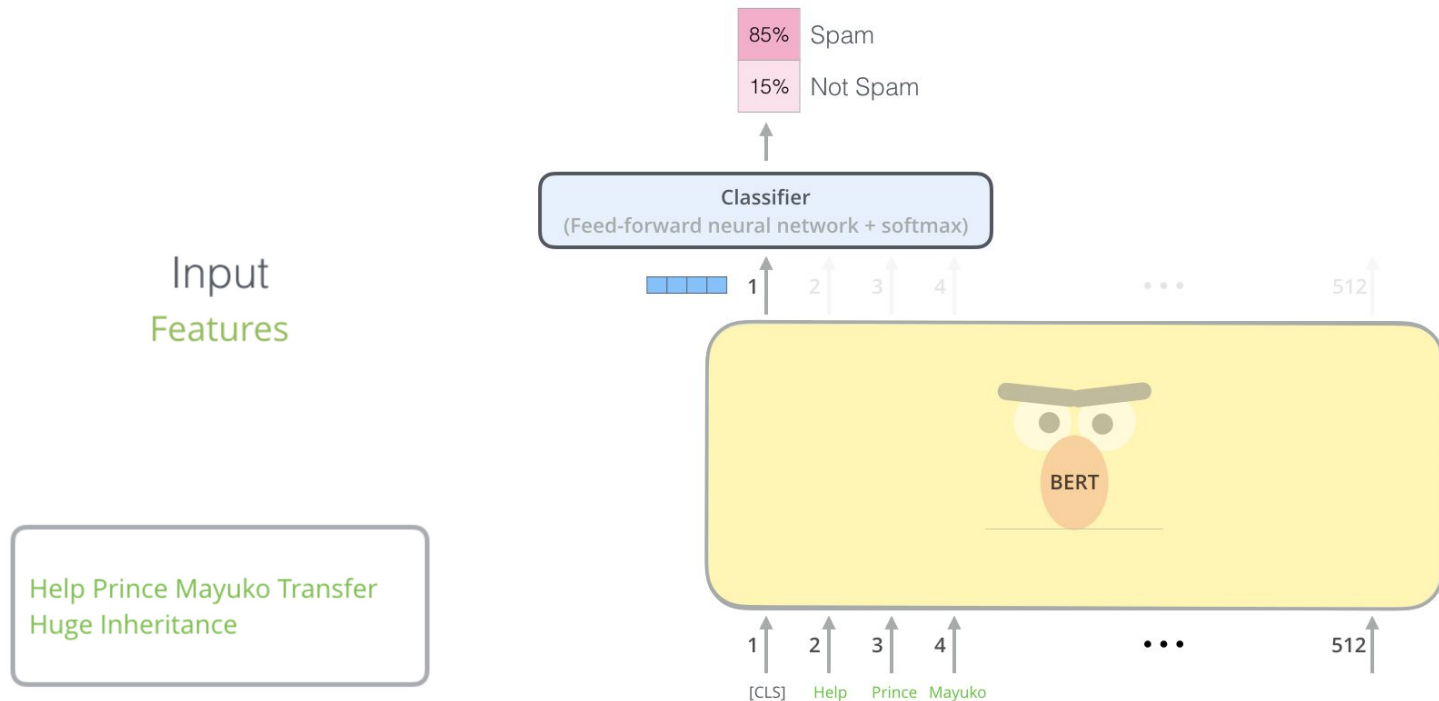
Use the output of the masked word's position to predict the masked word



Fine-tuning BERT

- BERT has learned rich representations which encode **syntax & semantics**
 - We can take advantage of this for “downstream” tasks via using **fine-tuning**
- Add & initialize a new layer on top of BERTs outputs
 - Use **supervised learning** to tune all parameters
- Because BERT is pretrained, fine-tuning is typically cheap
 - 3-4 epochs on 100s or 1000s of labelled examples
 - Typically takes a few minutes to a few hours to fine-tune on GPU(s)

Fine-tuning BERT



GLUE Test Results

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

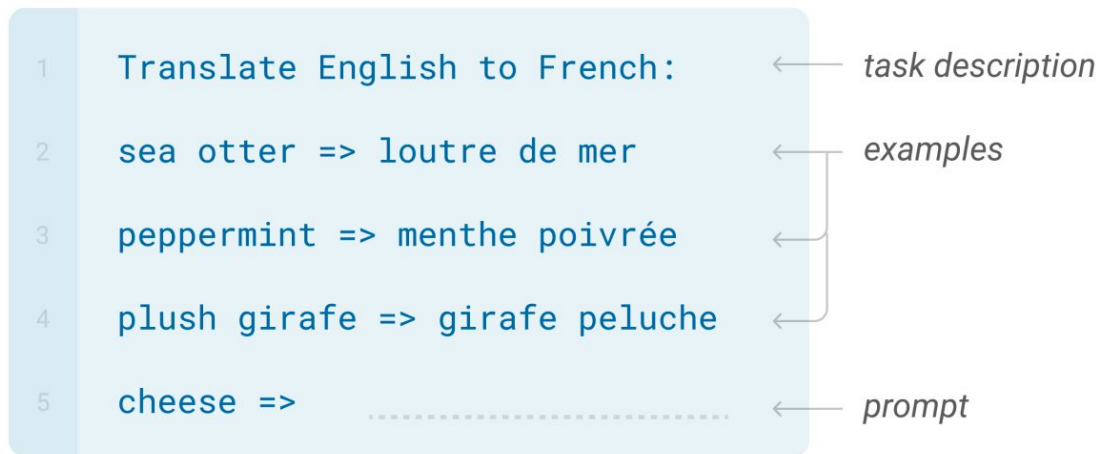
Table 1: GLUE Test results, scored by the evaluation server (<https://gluebenchmark.com/leaderboard>). The number below each task denotes the number of training examples. The “Average” column is slightly different than the official GLUE score, since we exclude the problematic WNLI set.⁸ BERT and OpenAI GPT are single-model, single task. F1 scores are reported for QQP and MRPC, Spearman correlations are reported for STS-B, and accuracy scores are reported for the other tasks. We exclude entries that use BERT as one of their components.

Modern (Large) Language Models – Prompting

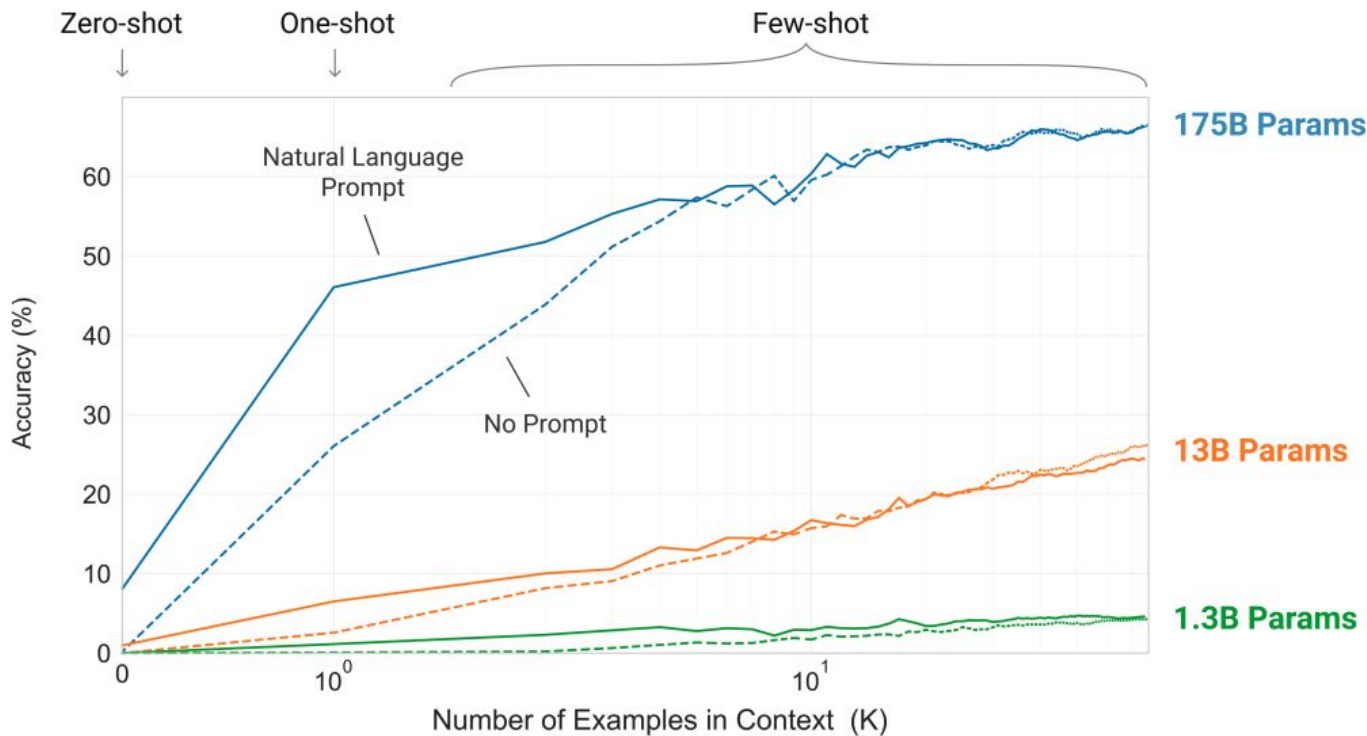
- Training specific output heads for a downstream task is no longer a popular approach
- Casting downstream tasks as text-to-text tasks allows for information to be shared between tasks (*transfer learning*)
 - Learn general representations for different concepts within the text

In-Context Learning

- Language modelling requires taking *context into account*
- Can we exploit that ability to use context and **provide “training” data in the context?**



In-Context Learning



Source: Brown, Tom, et al. "Language models are few-shot learners." Advances in neural information processing systems 33 (2020): 1877-1901.

Chain-of-Thought Prompting

- Examples can be made more helpful if they also *show to solve the problem*
- Encourages the model to put useful information in context

Standard Prompting	Chain-of-Thought Prompting
Model Input Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? A: The answer is 11. Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?	Model Input Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11. Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?
Model Output A: The answer is 27. ❌	Model Output A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Fine-Tuning Is Not Dead!

- Language on the internet rarely looks like an ICL prompt...
- Finetune on a set of examples of ICL!

	Meta-training	Inference
Task	C meta-training tasks	An unseen <i>target</i> task
Data given	Training examples $\mathcal{T}_i = \{(x_j^i, y_j^i)\}_{j=1}^{N_i}, \forall i \in [1, C] \quad (N_i \gg k)$	Training examples $(x_1, y_1), \dots, (x_k, y_k)$, Test input x
Objective	For each iteration, 1. Sample task $i \in [1, C]$ 2. Sample $k + 1$ examples from $\mathcal{T}_i$: $(x_1, y_1), \dots, (x_{k+1}, y_{k+1})$ 3. Maximize $P(y_{k+1} x_1, y_1, \dots, x_k, y_k, x_{k+1})$	$\operatorname{argmax}_{c \in \mathcal{C}} P(c x_1, y_1, \dots, x_k, y_k, x)$

Table 1: Overview of MetaICL (Section 3). MetaICL uses the same in-context learning setup at both meta-training and inference. At meta-training time, $k + 1$ examples for a task is sampled, where the last example acts as the test example and the rest k examples act as the training examples. Inference is the same as typical in-context learning where k labeled examples are used to make a prediction for a test input.

Fine-Tuning Is Not Dead!

- In general, text on the internet is *different from how people interact with an LLM*
- Fine-tune the LLM on input-output pairs resembling your expected interactions
 - **Instruction Tuning/Supervised Fine-Tuning (SFT)**
- Tune the model to produce preferred outputs based on human annotations
 - **Reinforcement Learning from Human Feedback (RLHF)**
- **You'll learn more about these later**

Resources

Some excellent resources this tutorial was based on

- [Speech and Language Processing](#) (great & free NLP textbook)
- [LSTM is dead. Long Live Transformers](#) (talk on why transformers > LSTMs)
- [Attention and Augmented Recurrent Neural Networks](#) (excellent blog post)
- [Attention Is All You Need](#) (paper that originally introduced transformers)
- <https://jalammar.github.io/> (excellent blog posts on NLP, Transformers, & more)

Follow-up Work

- NSP objective is typically discarded newer methods
 - A **Robustly Optimized BERT Pretraining Approach (RoBERTa)** drops NSP without harming downstream performance
 - **A Lite BERT** for Self-supervised Learning of Language Representations (**ALBERT**) argues that NSP is too easy (BERT gets 98% accuracy on it)
- BERT masks 15% of tokens in its input, but this is less than optimal
 - Wettig, Alexander, et al. (2022) improve performance by masking up to 40% of tokens!